

PROJSHARE

让 AI 学会下六洲棋

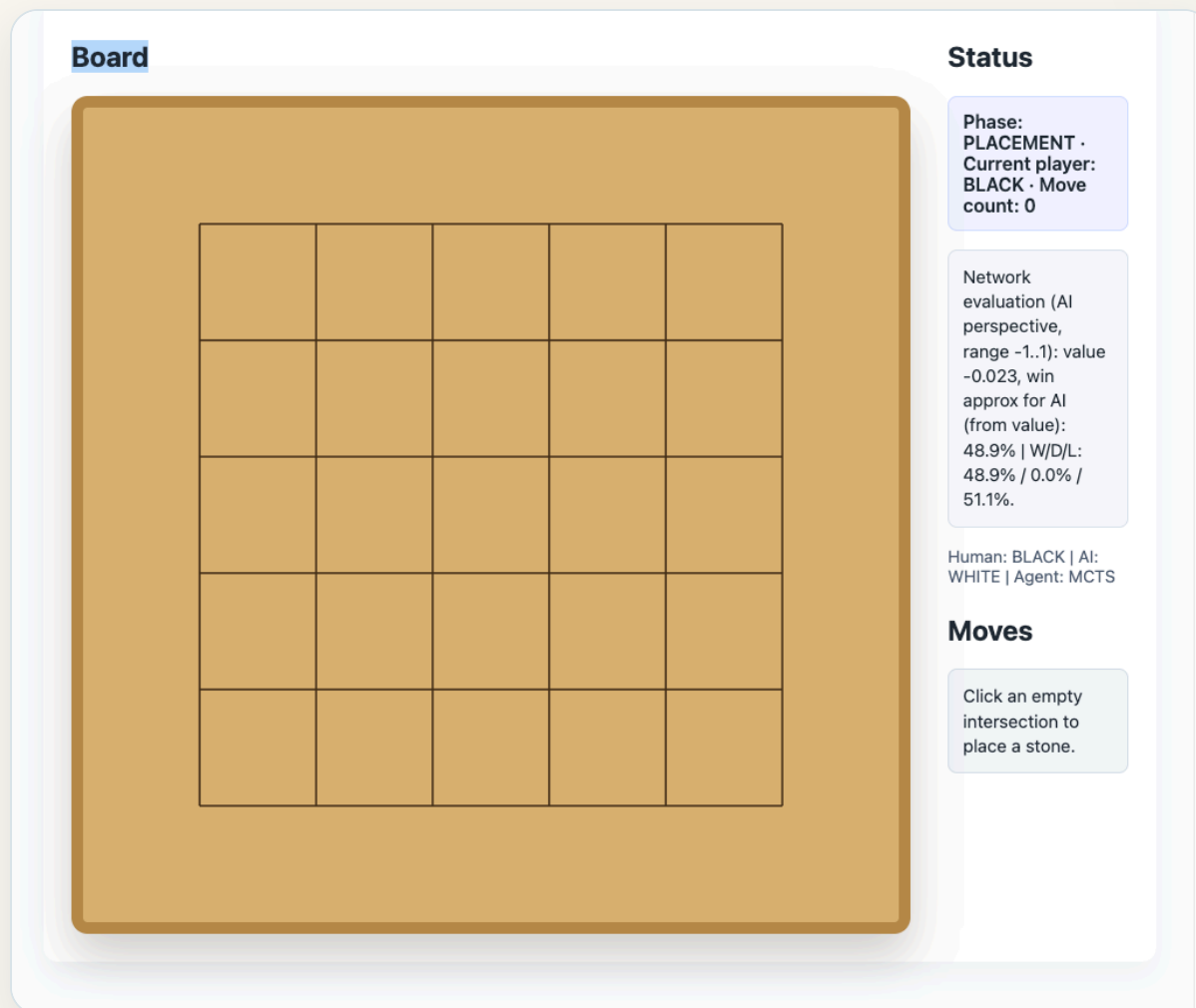
从规则、完整树搜索到自博弈训练：程序怎样一步步学会做选择

汇报人：张存远 · [2026-07-24]

目录

- 01 六洲棋规则背景** 初始棋盘、四阶段，以及“方/洲”
- 02 完整 MCTS** 是什么、为什么需要，以及怎样选择路线
- 03 策略符号与训练逻辑** s、p、v、 π 、z 分别是什么
- 04 PUCT** 如何兼顾已有证据与继续探索
- 05 代码与训练闭环** 伪代码、真实实现和网络结构

01 | 什么是六洲棋?



六洲棋规则

① 落子

双方轮流把棋子放到空位，直到棋盘填满。

② 形成“方/洲”后标记

落子形成方可标记 1 子；形成洲可标记 2 子。

③ 移除

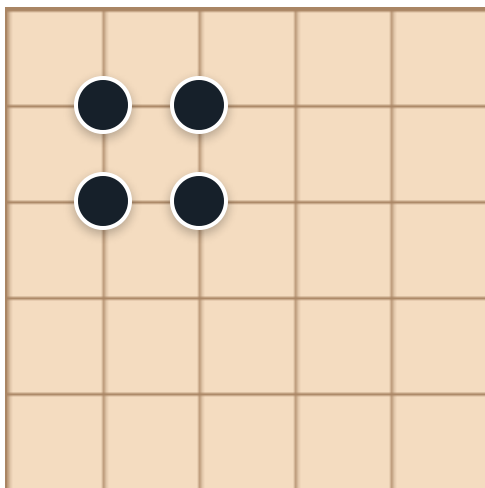
棋盘填满后，双方所有被标记的棋子一起移除。

④ 走子与提吃

棋子上下左右走一格；再形成方或洲时直接提吃。

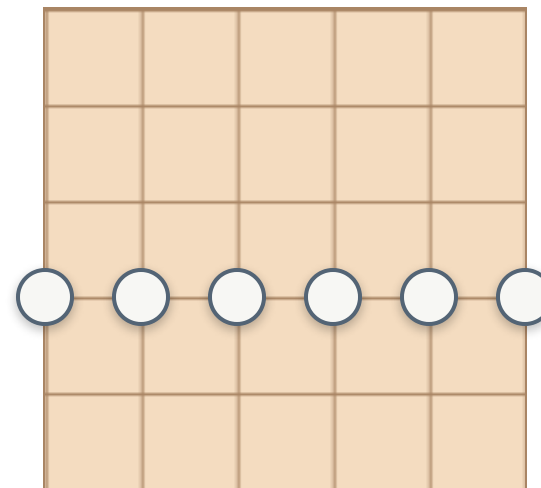
"方"和"洲"是规则的开关

方：2 × 2 同色棋子



落子阶段：标记对方 **1** 子
走子阶段：提吃对方 **1** 子

洲：一整行或一整列同色



落子阶段：标记对方 **2** 子
走子阶段：提吃对方 **2** 子

02 | 什么是 MCTS?

MONTE CARLO TREE SEARCH · 蒙特卡洛树搜索

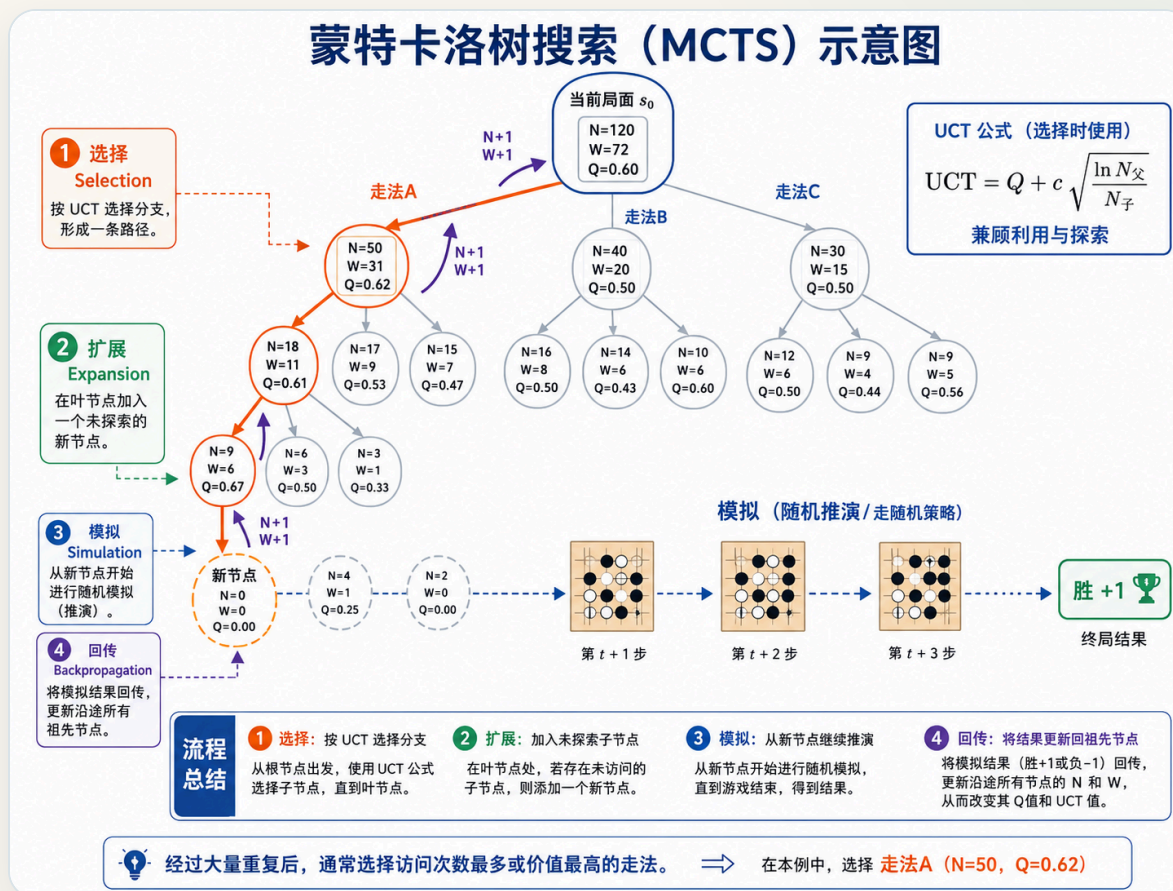
一种“边搜索、边建树”的决策算法

棋局可以画成一棵树：当前局面是根，一步棋是一条边，走完后的局面是子节点。

不把所有可能都搜索完，而是把计算资源集中到**更有希望的走法**上，通过反复试走判断哪一步更好。

核心循环：选择 → 扩展 → 模拟 / 评估 → 回传

蒙特卡洛树搜索 (MCTS) 示意图



完整 MCTS 的四步循环



重复多次后，重点路线会越搜越深；根节点的访问次数告诉我们最后更愿意走哪一步。

利用 | 探索

候选路线	当前观察	搜索充分吗	探索收益	本轮倾向
A：稳定路线	表现较好	已模拟很多次	较小	如果综合分仍高，就继续深入
B：潜力路线	暂时略逊于 A	只试过少量次数	较大	优先探索，确认是否被低估
C：弱势路线	表现较差	也没有充分搜索	较小	暂不优先，把预算留给 A / B

利用 Exploitation

继续验证当前已经表现较好的路线，让估计更稳定、搜索更深入。

探索 Exploration

给“看起来有潜力、但还没试够”的路线一次证明自己的机会。

随着访问次数增加，探索收益会下降，算法会不断重新排序。其中的PUCT算法会把探索和利用做好平衡。

03 | 符号定义

$$(s, \pi, z)$$

s

当时看到了什么

当前棋盘、行动方、规则阶段和标记信息。
输入网络时会变成多张 6×6 特征平面。

π

搜索后应该怎样下

把根节点各合法动作的访问次数变成概率。
它是搜索修正后的策略标签。

z

这盘棋最后怎样

对局结束后才知道：从样本中行动方视角
看，胜 +1、和 0、负 -1。

读作：看到局面 s ，搜索建议按 π 下，而这盘棋最后的结果是 z 。

网络看到 s 后，输出什么？

$$f_{\theta}(s) = (p_{\theta}(\cdot | s), v_{\theta}(s))$$

p: 第一直觉

网络刚看完局面时，对每个动作给出的初始倾向。
非法动作先由 legal mask 屏蔽，再在合法动作上形成先验分布。

v: 长期结果估计

从当前行动方视角看，最终更可能赢、平还是输。
它评价的是整盘棋的长期结果，而非这个Action的价值。

网络负责快速给出直觉；完整树搜索负责检查这个直觉经不起后续变化。

符号分别

符号	什么时候得到	通俗理解	训练中做什么
p	网络刚看完 s	未经搜索的“第一直觉”	策略头当前的预测
π	对 s 搜索完成后	搜索改进后的“教师答案”	策略头要拟合的目标
z	整盘棋结束之后	事后知道的真实胜平负	价值头要拟合的目标

π vs p

π 已经结合网络先验、PUCT 探索、深层局面和规则后果。

z vs v

v 是当时的估计； z 是对局结束后回填的监督结果。

搜索和网络互相改进

只用网络

一次前向很快，但它不一定能准确看到几步之后的战术后果。

只用搜索

每个局面都从零开始尝试，没有先验引导，计算量会很大。

网络引导搜索

把有限预算集中到更有希望的动作。



搜索改进策略

把规则后果和向前试走融入 π 。



网络学习 π 与 z

把搜索结果“压回”一次前向。

理想情况下这是正反馈；如果 value 偏差大、搜索太弱或数据覆盖差，也可能变成自我确认。

04 | PUCT: 下一次该试哪条路?

$$\text{PUCT}(s, a) = Q(s, a) + c_{\text{puct}} P(s, a) \frac{\sqrt{N(s)}}{1 + N(s, a)}$$

Q: 搜索后的平均结果

深层回传值在这条边上的平均。

P: 模型的第一印象

节点展开时，网络给动作的先验。

N(s): 节点总访问量

当前节点被经过的总次数。

N(s,a): 边访问次数

这条边被经过了多少次。

Q 是搜索积累的证据

$$Q(s, a) = \frac{W(s, a)}{N(s, a)} = \frac{v_1 + v_2 + \cdots + v_{N(s, a)}}{N(s, a)}$$

动作刚被尝试时

Q 可能主要来自某次新叶子的网络估计 v 。

树继续向下生长后

更深后代的网络值和准确终局结果不断回传，Q 会持续重算。

所以 Q 表示“当前整棵搜索树对这条边的平均判断”。

同一个 PUCT 公式会在根节点和内部节点反复使用。

PUCT 为什么这样设计？

假设当前节点一共访问 16 次，且 $c = 1$ ：

候选走法	当前平均 Q	模型先验 P	已访问 $N(s,a)$	探索奖励 U	总分 $Q + U$
A：稳妥	0.62	0.12	15	0.03	0.65
B：有潜力	0.48	0.35	3	0.35	0.83
C：几乎未知	0.00	0.08	0	0.32	0.32

只看 Q

会一直选 A，可能错过真正更好的 B。

加入探索奖励 U

B 得到继续向深处验证的机会；之后的回传又会更新 Q。

05 | 完整树搜索：主循环伪代码

```
function FULL_MCTS(当前局面 s):  
    root = 建立节点(s)  
    用网络得到 p(root), v(root), 并展开 root  
  
    repeat simulation = 1 ... S:  
        path = [root]  
        node = root  
  
        while node 已展开, 并且不是终局:  
            action = argmax_a PUCT(node, a)  
            node = node.children[action]  
            path.append(node)  
  
        if node 是终局:  
            leaf_value = 准确的胜 / 平 / 负  
        else:  
            p_leaf, v_leaf = 网络(node.state)  
            用合法动作和 p_leaf 展开 node  
            leaf_value = v_leaf  
  
        沿 path 回传 leaf_value, 更新 N、W、Q  
  
     $\pi(a|s) \propto N(\text{root}, a)^{(1 / \text{temperature})}$   
    return  $\pi$ 
```

PUCT 作用在树的每一层；网络只评估新叶子；根访问分布才是训练标签 π 。

当前代码：沿完整树选择路径

```
def _select_path(self, root: PortableNode) -> List[PortableNode]:
    path = [root]
    node = root
    while node.expanded and node.children and not node.terminal:
        sqrt_total = math.sqrt(max(1, node.visit_count))
        best_score = -math.inf
        best_index = -1
        best_child: Optional[PortableNode] = None

        for action_index in sorted(node.children):
            child = node.children[action_index]
            q = value_for_parent(node, child) if child.visit_count > 0 else 0.0
            u = (
                float(self.config.exploration_weight)
                * float(child.prior)
                * sqrt_total
                / (1.0 + child.visit_count)
            )
            score = q + u
            if score > best_score or (
                score == best_score and action_index < best_index
            ):
                best_score = score
                best_index = action_index
                best_child = child

        if best_child is None:
            break
        node = best_child
        path.append(node)
    return path
```

来源： `v1/python/portable_mcts.py::_select_path` 。C++ 生产实现保持同一选择语义。

代码怎样接入一轮训练？

① 自博弈

当前模型 + 完整树 PUCT 生成 (s, π, z) 轨迹。



② 训练

让网络策略 p 拟合 π ，让价值 v 拟合终局与软价值目标。



③ 评估

对随机程序检查健康度，并与当前 best 对战。



④ 推进

保存 candidate；分别更新 latest 与 best。

脚本调度层： `scripts/big_train_v1.sh` / portable 长训编排 → `v1/train.py`

本节按当前 `search_backend=portable`、`portable_mcts_backend=cpp` 讲解：PUCT 在树的每一层选择， Q 由完整路径回传持续更新。

自博弈阶段：完整树怎样接入

```
v1/train.py
└─ v1/python/self_play_worker.py
    └─ portable_cpp_self_play.py
        └─ while 仍有未结束对局：
            └─ 当前状态编码为 model_input      (B, 11, 6, 6)
            └─ 规则生成 legal_mask            (B, 220)
            └─ PortableCppMCTS.search_batch()
                └─ 准备 / 复用每盘棋的搜索树
                └─ repeat S 次完整模拟：
                    └─ 每层按 PUCT 选择到叶子
                    └─ 批量网络评估叶子 p, v
                    └─ 用合法动作扩展叶子
                    └─ 沿完整路径回传，更新 N, W, Q
                └─  $\pi(a|s) \propto N(a)^{(1 / \text{temperature})}$ 
                └─ 从  $\pi$  采样或选择动作
```

动作执行后，搜索树会把被选中的子树提升为新根，继续复用已经得到的信息。

一盘结束后，怎样形成训练数据？

state_tensors

每一步网络实际看到的状态

(N, 11, 6, 6)

legal_masks

该阶段允许的 220 维动作

(N, 220)

policy_targets

完整树根节点访问分布 π

(N, 220)

value_targets

终局胜平负 z，换成该步行动方视角

(N,)

soft_value_targets

终局棋子差提供的辅助局面信号

(N,)

先用黑方统一记录结果

`result_from_black`：黑胜 +1、和 0、白胜 -1。

再按每步行动方换视角

`z = result_from_black × player_sign`。

训练阶段：p 学 π , v 学目标

```
v1/python/train_bridge.py
└─ for epoch; for batch:
    │ 网络前向:
    │   log_p1, log_p2, log_pmc, value_logits = model(states)
    │ 三张 6×6 策略图组合为 220 维 combined_logits
    │ legal mask 后做 log-softmax
    │ policy_loss = CrossEntropy / KL( $\pi$ _search || p_network)
    │
    │ mixed_value = (1 -  $\lambda$ ) · hard_z +  $\lambda$  · soft_value
    │ mixed_value → 101 桶 two-hot target
    │ value_loss = bucket cross-entropy
    │
    │ total_loss = policy_loss + value_loss
    └─ backward → gradient clip → Adam step
```

策略头

学完整树搜索后的 π ，不是实际落子的 one-hot。

价值头

主目标是终局结果，同时可混合局面软信号。

连续训练

可恢复上一轮 Adam 状态，避免每轮重新冷启动。

评估与模型推进: latest $\neq$ best

latest model

本轮训练得到的 candidate。只要数值有效，下一轮自博弈通常继续从它出发。

代表“训练走到哪里了”

best model

candidate 与 incumbent 对战并达到门槛后，才会替换。

代表“目前保留下来的最好模型”

vs Random

检查模型是否正常进步，不作为最终棋力结论。

vs Best Previous

决定 candidate 是否晋升为新的 best。

Tournament / Elo

需要比较多个强模型时，再做相对强度排序。

网络输入：11 张 6×6 特征图

$$X \in \mathbb{R}^{B \times 11 \times 6 \times 6}$$

自己的棋子

channel 0

对手的棋子

channel 1

自己的标记

channel 2

对手的标记

channel 3

7 个阶段平面

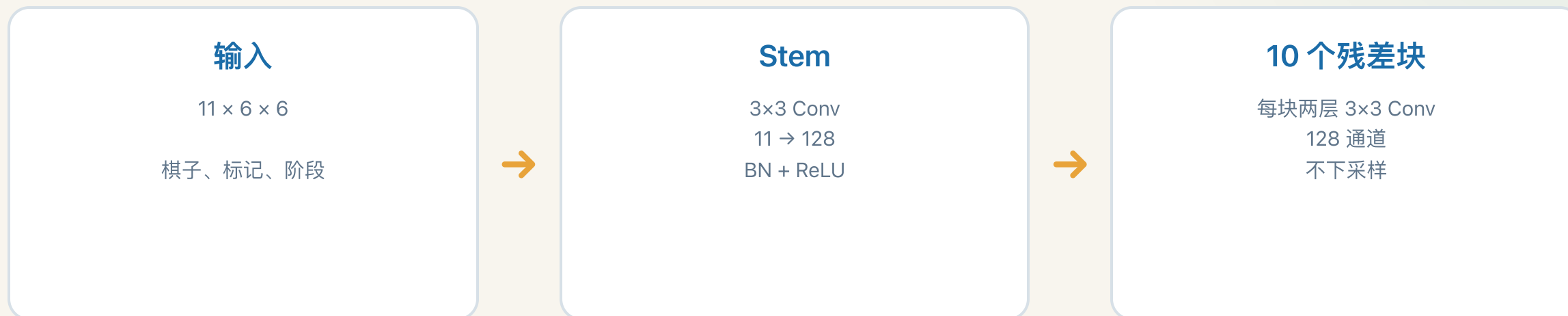
PLACEMENT / MARK / REMOVAL / MOVEMENT / CAPTURE / FORCED / COUNTER

相对行动方

轮到白方时交换 self / opponent 通道

它不是 RGB 图片，而是一张很小、每个通道都有明确含义的“多层棋盘图”。

共享残差主干：始终保持 6×6



为什么不缩小棋盘?

棋盘只有 6×6，保留空间分辨率可以避免丢掉精确落点。

为什么策略和价值共享主干?

二者都需要理解相邻关系、方、洲、空位和标记。

Stem 加第一个残差块已有 3 层 3×3 卷积，理论感受野 7×7，已经覆盖整个 6×6 棋盘。

两个输出头：一个管“怎么走”，一个管“局面怎样”

Policy Head

128×6×6
→ 1×1 Conv: 128→64
→ GlobalPool: mean/max/std
→ 全局信息加回局部特征
→ 三张 6×6 heatmap
 p1 / p2 / pmc
→ 动作编码 + legal mask
→ 220 维动作策略

不是直接输出一条 220 维向量，而是先按动作语义生成三张棋盘热力图。

Value Head

128×6×6
→ 1×1 Conv: 128→64
→ GlobalPool: 64→192
→ Linear: 192→128
→ Linear: 128→101
→ 101 个 [-1,1] 价值桶
→ softmax 加权求期望得到 v

价值头关心整盘结果，所以通过全局池化汇总整个棋盘。

为什么网络这样设计？

小棋盘不下采样

始终保持 6×6，精确位置不丢失。

阶段作为条件

同一棋盘在不同规则阶段，可以得到不同判断。

策略按动作语义拆分

落子、移动、标记与提吃共享棋盘表征，但评分方式不同。

价值输出分布

101 桶比单个标量保留更多不确定性信息。

一句话：主干先回答“棋盘上发生了什么”，两个头再分别回答“怎样走”和“最后会怎样”。

从“会走”到“会学习”

规则定义边界 $\rightarrow$ 网络给出 p 、 v $\rightarrow$ 完整树 PUCT 得到 π $\rightarrow$ 终局回填 z

六洲棋 AI 的核心，不是记住一套固定走法，
而是让搜索产生更好的老师，再让网络把老师的经验学回去。

谢谢 · Q & A